

I-HSUAN(ETHAN) HUANG

631-605-5130 | ih246@cornell.edu | ethancornell.github.io | github.com/EthanCornell

PROFESSIONAL SUMMARY

Software engineer with expertise in concurrency, system optimization, and backend development. Open to junior-to-mid level roles where I can apply my skills in large-scale distributed systems, high-performance computing, operating systems, or backend infrastructure, while continuing to grow in a fast-paced engineering environment. Actively expanding expertise in large-scale distributed systems, high-performance computing, and advanced backend infrastructure. Excited to collaborate with experienced engineers and contribute to cutting-edge innovations in cloud computing, AI systems, and low-latency architectures.

EDUCATIONAL BACKGROUND

Cornell University, New York, NY
Master of Engineering in Computer Science

Aug. 2022 - May 2023

TECHNICAL PROFICIENCIES

Languages:	C++11/17, C11, Java, Python, Shell Script, JavaScript, PHP
OS, Database, Compiler:	Linux, FreeBSD, UNIX, MacOS, Windows, MySQL, PostgreSQL, SQL, GCC, g++, JVM
Tools, API:	Git, GDB, Valgrind, gprof, GCP, Vim, STL, OpenMP, OpenMPI, CUDA, PyTorch
Skills:	Algorithms, Data Structure, Parallel/Concurrency Programming, Memory Management

RELEVANT EXPERIENCE

Cornell University, Research Assistant, Ithaca, NY
Advised by Professor Robbert van Renesse

Aug. 2023 - Present

- Engineered a sanitizer-clean, header-only [Michael-Scott MPMC queue in C++20 with hybrid memory reclamation \(hazard pointers + 3-epoch EBR\)](#), delivering 1.7 M+ ops/sec single-threaded, 2 M+ ops/sec at 16 threads, and zero ABA/use-after-free safety under high contention.
- Designed and delivered a crash-safe, [log-structured file system](#) (4 KB blocks, checkpoint + imap) with a UDP server, POSIX-style client library, one-command CI test harness, and on-disk tracing tools—self-formats on first boot, reloads state in < 1 ms after crashes, and passes every functional & recovery test.
- Built a [cross-platform file-migration tool](#) with resumable, block-level incremental backup that refreshes a 50 MB file after a mere 16 KB change in < 1 s, maintains 100 % data integrity through crash/pause/network-fault tests.
- Optimized runtime performance and scalability of the [Harmony model checker](#) by designing and implementing sharding techniques and lock-free data structures, enhancing concurrent processing and reducing CPU contention.
- Addressed load imbalances and synchronization inefficiencies by refining hash table management strategies and synchronization protocols, resulting in a 7% performance boost on a 64-core CPU server.
- Developed and implemented a [fault-tolerant distributed chat server](#) using C++17, with the Raft consensus algorithm for leader election, log replication, and state machine consistency, achieving high availability and robust network communication.
- Enhanced C++17 [lock-free linked lists](#) with OpenMP, achieving a 9.5x ops/ms improvement and integrating visual analytics support for performance monitoring.
- [Optimized C++ singly linked list](#) performance by implementing advanced smart pointer techniques, achieving up to 5x speed improvement over the STL forward list.
- Improved multi-threaded system performance through advanced cache management and thread-safe operations, significantly reducing disk I/O and enhancing overall system efficiency.
- Enhanced memory management and system performance by developing concurrent and sequential data structures and parallel algorithms in C/C++ and Python, resulting in improved computational efficiency, with project management and code integration via [GitHub](#).
- Enhanced system stability and reliability in a multi-threaded environment by integrating pthread mutexes for critical cache operations, conducting extensive testing to ensure deadlock prevention and maintain operational integrity.
- Designed and optimized [cache replacement strategies](#) in C, including a groundbreaking LFRU algorithm achieving a remarkable 99.1% peak hit rate in resource-constrained scenarios.
- Developed a Thread Management System in C, improving system performance and responsiveness by efficiently managing thread life cycles and resource allocation.
- Enhanced multi-threading in a C-based project by optimizing thread life-cycle management and implementing semaphore-based synchronization, leading to improved system concurrency and reduced latency.
- Developed an advanced thread management framework with optimized semaphore operations, boosting performance and ensuring stability in multi-threaded environments.
- Implemented Low-Latency Zero-Copy Techniques in the RISC-V [cgos2000](#) file system, reducing latency and enhancing write operation performance by optimizing data flow and minimizing CPU overhead.

Advised by Professor Robbert van Renesse

- Streamlined RISC-V egos-2000 OS by introducing a two-level page table and Soft TLB, significantly enhancing MMU robustness, improving overall memory efficiency, and reducing the risk of errors.
- Contributed to operating system kernel development by enhancing process scheduling, exception/interrupt handling, and IPC in C, which led to noticeable improvements in system performance and reliability.

Viking Family International Ltd., Founding Software Engineer(Backend), Taipei, Taiwan

Nov. 2014 - Aug. 2021

Led the development of an e-commerce platform utilizing **Magento** for a Taiwan-based wholesale tea supplier, streamlining online sales, distribution, and import services for clients including Lady M Taiwan, Burberry Taiwan, IBM Taiwan, and Mandarin Oriental Taipei. Oversaw the full software development lifecycle, from **architecture design to deployment**, enhancing **customer experience and operational efficiency**.

- Optimized warehouse and inventory management** by integrating a multi-threaded order processing system using **C++ mutexes**, significantly reducing processing times and improving fulfillment accuracy.
- Refactored an accounting web application in Java**, applying **data structure optimization** to improve **server response time by 11%**.
- Conducted sales data analysis in Python**, extracting key business insights that led to a **2% revenue increase** through data-driven decision-making.
- Designed and implemented a **real-time inventory synchronization system** using **Odoo ERP**, ensuring **data consistency** across the e-commerce platform, warehouse, and POS systems.
- Applied **concurrent programming techniques** such as **optimistic/pessimistic locking** and **mutex-based synchronization** to prevent race conditions, ensuring **accurate stock management**.
- Developed **ACID-compliant transaction handling** for stock reservations and order processing, boosting system **reliability and customer trust** during high-traffic periods.
- Built an **event-driven notification system** using **Apache Kafka**, allowing efficient stock updates and ensuring **accurate reservations** during the checkout process.
- Enhanced system resilience by implementing **idempotent operations, retry mechanisms, and dead letter queues**, improving **error handling and service reliability**.

PCPartner Group Ltd., Software Engineer, Taipei, Taiwan

Aug. 2012 - Jul. 2014

- Developed an Android media streaming app with seamless integration for DLNA-certified devices, enhancing user experience and expanding compatibility with home media systems.
- Implemented UPnP/UDP Library integration using Java and C, enriching media streaming app features by improving device connectivity and streaming performance.

ASUSTek Computer Inc., Software R&D Engineer, Taipei, Taiwan

Jul. 2011 - Jul. 2012

- Enhanced Android OS development by creating a visual CI pipeline with Python and Shell, cutting development time by 60% and increasing team productivity by up to 300%.
- Contributed to Android OS debugging using C, identifying and resolving critical system bugs that improved OS stability and performance.

OPEN SOURCE EXPERIENCES**FreeBSD**, Individual Contributor | C, GCC9, GDB

Apr. 2024 - Present

[Enhanced FreeBSD netgraph concurrency](#) by implementing lock-free C structures, significantly boosting network throughput and reducing latency, thereby improving overall system responsiveness under heavy network loads.

- Optimized Netgraph Concurrency:** Improved lock-free traffic handling within a 2-node topology, effectively reducing latency and increasing throughput, leading to more efficient data flow in high-demand scenarios.
- Engineered Lock-Free Data Structures:** Developed and integrated lockless data structures, utilizing C and epoch-based memory management, to remove common bottlenecks associated with mutexes and locking in FreeBSD's netgraph nodes.
- Enhanced Low-Latency Traffic Flow:** Designed concurrent data handling mechanisms, reducing contention, and improving network efficiency during high-load conditions, which ensured stable performance.
- C Programming for System Optimization:** Applied advanced C programming and systems optimization techniques to resolve concurrency challenges, maintaining robust network performance and ensuring stable operations in multi-threaded environments.
- Collaborated with senior FreeBSD contributors, including engineers from Netflix, to review design approaches, validate concurrency strategies, and discuss performance trade-offs related to epoch-based memory management in Netgraph.